
UNIT 1 DATABASE MANAGEMENT SYSTEM – AN INTRODUCTION

Structure

- 1.0 Introduction
- 1.1 Objectives
- 1.2 Need for a Database Management System
 - 1.2.1 The File Based System
 - 1.2.2 Limitations of File Based System
 - 1.2.3 The Database Approach
- 1.3 Logical DBMS Architecture
 - 1.3.1 Three Level Architecture of DBMS
 - 1.3.2 Mappings between Levels and Data Independence
 - 1.3.3 The Need of Three Level Architecture
- 1.4 Physical DBMS Structure
 - 1.4.1 DML Precompiler
 - 1.4.2 DDL Compiler
 - 1.4.3 File Manager
 - 1.4.4 Database Manager
 - 1.4.5 Query Processor
 - 1.4.6 Database Administrator
 - 1.4.7 Data files, indices and Data Dictionary
- 1.5 Database System Architectures
- 1.6 Data Models and Trends
- 1.7 Summary
- 1.8 Solutions/Answers

1.0 INTRODUCTION

In the present time, most of your online activities require interaction with a database system running as the backend of an application, such as purchasing from supermarkets or e-commerce website, depositing and/or withdrawing from a bank, booking hotel, airline or railway reservation, accessing a computerised library, ordering a magazine subscription from a publisher, using your smartphone apps to purchase goods. In all the above cases a database is accessed. Most of these backend database systems may be called Traditional Database Applications. In these types of databases, the information stored and accessed is textual or numeric. However, with advances in technology in the past decades, different newer database models have been developed, which will be discussed in Block 4 of this course. In this unit, we will be introducing the architecture and structure of a traditional Database Management System.

1.1 OBJECTIVES

After going through this unit, you should be able to:

- describe the File Based system and its limitations;
- describe the structure of DBMS;
- define the functions of DBA;
- explain the three-tier architecture of DBMS and its need; and
- appreciate different database models.

1.2 NEED FOR A DATABASE MANAGEMENT SYSTEM

A Database is an organised, persistent collection of data of an organisation. The database management system manages the database of an enterprise. Prior to use of the database systems, file-based systems were popularly used. In order to appreciate the strengths of database management systems, you may first list the problems associated with the file base systems, which are discussed next.

1.2.1 File Based System

Computerised file-based systems were primarily designed to make electronic versions of physical filing systems used by businesses. For example, a physical file can be set up to hold all the correspondence relating to a particular matter of a project, product, task, client or employee. In an organisation there could be many such files, which may be labeled and stored. Similarly, at homes you may have to maintain files relating to bank statements, loans, receipts, tax payments, etc. You can use electronic means to create these files. How do you search specific information from these files? For finding information, the entries could be searched sequentially. Alternatively, an indexing system could be used to locate the information directly, for example, you search for specific content in different files in your computer file search options.

The filing system works well when the number of items to be stored is small. It even works quite well when the number of items stored is quite large and they are only needed to be stored. However, a manual file system crashes when cross-referencing and processing of information in the files is carried out. For example, the University has several students who can register for different programmes of the University. The University has several faculty members who teach different courses to the enrolled students. The university may have to maintain separate files for the personal details of students, fees paid by them, and the details of the courses undertaken by them. In addition, the University also needs files for keeping details of each faculty member in various departments, courses taught by them. How would the following queries be answered?

- Fee paid by the students of the Computer Science Department per annum.
- The students who are willing to avail the pickup bus facility.
- Number of students taught by a faculty in a specific academic period.
- The number of students who have passed the programme this year in comparison to earlier years.
- How many students of a specific department have registered for courses other than their own department?

Please refer to *Figure 1*. The answer to all the questions cannot be computed by just simple statements but will require extensive file processing. Thus, each of these queries will require substantial amount of time in the file-based system.

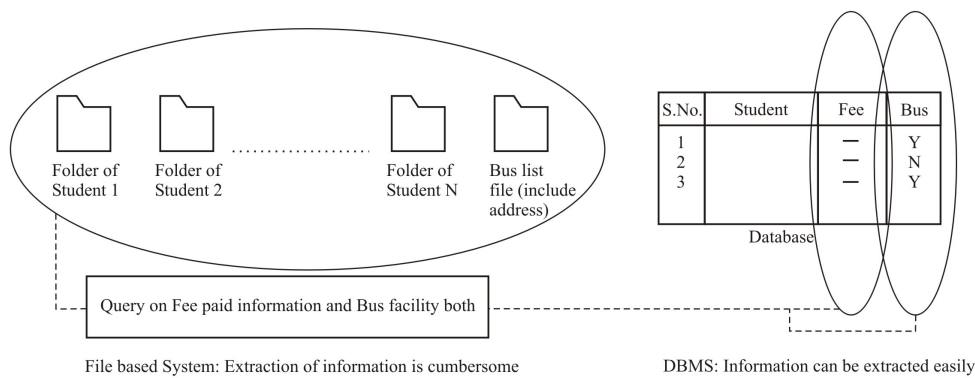


Figure 1: Information extraction using a file based system

1.2.2 Limitations of File Based System

File based systems required that several files should be opened for a particular system application. Several files may consist of duplicate data, which can result in several shortcomings. Some of these shortcomings are listed below:

- **Data isolation:** Since the file system stores data in separate files, which may belong to different applications. These files are not accessible to other applications and are difficult to share, especially when an application needs to use more than one file. Also, as the number of files can be very large for such systems, therefore, it would be difficult to search the relevant data from these files.
- **Data Duplication:** As stated earlier, a file system has different files for different applications, which may have overlapping data requirements. This will result in duplication of data, which can result in inconsistent data when duplicate data is updated. In addition, data duplication can also result in waste of storage. An example of data duplication is shown in Figure 1, where the address of several students may be stored in two different files, viz. student folder and bus list file.
- **Inconsistent Data:** The data in a file system can become inconsistent if more than one person modifies the data concurrently, for example, if any student changes the residence and the change is notified to only his/her file and not to the bus list. Entering wrong data is also another reason for inconsistencies.
- **Data dependence:** In the file systems, you need to clearly define the storage organisation of data files and the structure of the records in the application code. This means that it is extremely difficult to make changes to the existing structure, as any change in structure would require change in all the programs using that structure of the data. The programmer would have to identify all the affected programs, modify them and retest them. This characteristic of the File Based system is called **program data dependence**.
- **Incompatible File Formats:** Since the structure of the files is embedded in application programs, the structure is dependent on application programming languages. Hence the structure of a file generated by COBOL programming language may be quite different from a file generated by 'C' programming language. This incompatibility makes them difficult to process jointly. The application developer may have to develop software to convert the files to some common format for processing. However, this may be time consuming and expensive.
- **Fixed Queries:** File based systems are very much dependent on application programs. Any query or report needed by the organization would require the

application programmer to write a new program. As the type and number of queries or reports are expected to increase with time, producing different types of queries or reports would be difficult to implement in file-based systems. Since applications of file-based systems are designed to answer specific queries, therefore, new queries cannot be answered without generating an application. This entire process is time consuming and complex.

The file-based systems require large number of applications, therefore, are difficult to maintain. Further, each application would require separate provision from security. Besides the above, the maintenance of the File Based System is difficult and there is no provision for security. Further, data recovery from failures is inadequate or non-existent.

1.2.3 The Database Approach

As discussed in the previous section, the file system has many weaknesses. Therefore, a new approach was proposed that eliminates the weaknesses of the file system. This approach, called the database approach, separated data from application programs. The data in this approach is integrated from various applications and securely shared using a management system. *A database stores the integrated data of an organisation in a persistent manner.* The following are some of the characteristics of the database approach:

- The database can store data in a centralised database or a distributed database.
- The data is managed by a database management system (DBMS)
- It contains additional data about data, called metadata, which describes the structure and constraints on the stored data. The metadata is stored in DBMS in a data dictionary or system catalog.
- The database integrates the data of an organisation. This data is shared under the control of DBMS in a secure manner.
- DBMS allows several basic operations related to data, such as creating a database structure, inserting and editing data in a database, enforcing security and constraints on data and allowing access to data to authorised users.

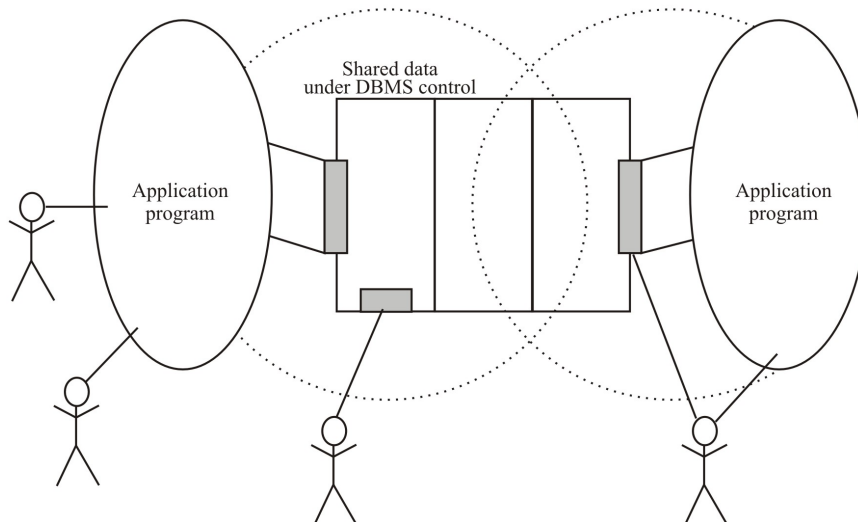
Let us discuss these advantages of database systems in more detail.

Reduction of Redundancies

The data of a database system is integrated, therefore, removes any duplicate data, as was the case of a file-based system, which maintains different files for different applications. The files that contain the copy of the data may become inconsistent as some of the files may be updated, whereas others may not be. The reduction in duplicate data may also help in reducing such data inconsistencies among the duplicated data of files. Thus, in the database approach, data is stored at a single place or with controlled redundancy under the control of DBMS, which helps in reducing data inconsistency.

Sharing of Data

The data of a database is integrated data of the entire organisation, however, the authorised users may be permitted to share partial data (why?). This scheme can be best explained with the help of a logical diagram (*Figure 2*). New applications can be built and added to the current system and data can be shared with these data, as per the need of such applications. Please note that data sharing is controlled by the DBMS, such that only the authorised users or applications can access it.



A user can either access data window through DBMS or use an application program.

Figure 2: User interaction to data through DBMS

Data Independence

In the file-based system, the descriptions of data and logic for accessing the data are built into each application program making the program more dependent on data. A change in the structure of data may require alterations to programs. Database Management systems separate data descriptions from data. Hence it is not affected by changes. This is called Data Independence, where details of data are not exposed. DBMS provides an abstract view and hides details. For example, in Figure 2, you can observe that the interface or window to data provided by DBMS to a user may still be the same although the internal structure of the data is changed.

Improved Integrity

Data Integrity refers to validity and consistency of data. Data Integrity means that the data should be accurate and consistent. This is implemented by enforcing checks or constraints on the data while it is being entered or manipulated in a database. Data of a database is not allowed to violate these constraints or rules. Constraints may apply to data items within a record or relationships between records. For example, a constraint on the age of an employee can be between 18 and 70 years only. While entering or modifying the data of the age of an employee, the DBMS should enforce this constraint. However, please note there is still a possible error if you enter the age of a person as 55 instead of 25. Such errors would require different ways of checking. Database systems support many other types of constraints, which will be discussed in later units.

Efficient Data Access

DBMS utilises techniques to store and retrieve the data efficiently, at least for unforeseen queries. An advanced DBMS allows its users to access data efficiently.

User Interfaces as per Users technical knowledge

A DBMS allows different types of interfaces, based on different levels of their technical knowledge. For example, the following types of interfaces may be provided by a DBMS:

- Menu driven forms and reports-based interfaces
- Application programming interface
- Query language interfaces
- Natural language interfaces

Representing relationship among data

Data of a database system is integrated data of an organization, which includes large number of related data objects. DBMS should maintain and preserve these relationships so that related data can be accessed easily.

Improved Security

The data of an organisation is vital and confidential. DBMS allows users to share only that information that they are authorised to access. For example, the critical information of an employee of an organisation should not be accessible to any other employee of that organisation. Hence, data of the database should be protected from unauthorised users. This is implemented by Database Administrator (DBA), who provides the users with controlled privileges on the data and type of operations. To enforce security, DBMS has a security and authorisation subsystem. Only authorised users may use the specific data of a database. Further, the operations performed by these users on data can be restricted to data retrieval, insertion, update, deletion etc. or any combination of these operations. For example, the Branch Manager of any company may have access to all data and is allowed to modify the incentive data of employees, whereas the Sales Assistant may not have access to salary details.

Improved Backup and Recovery

A file-based system may fail to provide measures to protect data from system failures, as taking backups periodically is the responsibility of the user. However, most of the DBMSs are designed in such a manner that it can recover from different types of hardware failures. In addition, DBMS also supports data backup. In general, a DBMS has a subsystem to support such provisions.

Support for concurrent Transactions

A transaction, in the context of a database system, is an atomic operation that is either completed fully or not at all. A database should allow multiple transactions to be carried out at the same time. For example, in a bank several money withdrawal and money transfer operations may be carried out at the same time. Most of the commercial DBMSs ensure that all these transactions do not interfere with each other.

Check Your Progress 1

- 1) What is a DBMS?
.....
.....
- 2) What are the advantages of a DBMS?
.....
.....
- 3) Compare and contrast the traditional File based system with Database approach.
.....
.....

1.3 THE LOGICAL DBMS ARCHITECTURE

As discussed in the previous section that most of the advantages of database systems are because of the creation of DBMS software. DBMSs must support many services and therefore are complex in nature. In addition, DBMSs are also required to store, manipulate and control a very large amount of data in a reliable manner. In this and subsequent section, we discuss the architecture of DBMS, which will help you with the processes and features of a DBMS.

In this section, two different architectures, which deal with two different aspects of database management, are discussed. The first architecture is the logical architecture,

which defines the data organisation and access at different logical levels of a database. The second architecture defines various components of a DBMS software. This architecture is referred to as physical database architecture.

1.3.1 Three Level Database Architecture

The three-level database architecture of a database defines the three different levels of abstraction of data for different types of users of the database. The proposed architecture was designed and standardised by the American National Standards Institute (ANSI) and is also known as ANSI/SPARC architecture. As per this architecture, a database schema can be visualised at three different levels. Figure 3 shows these three levels of this architecture. These levels are explained next.

The External or View Level

This level of abstraction provides a view of data for the users of a database system. Typically, this abstraction is created based on access rights of the users. Different types of users can be allowed different external views of data, as shown in Figure 3. Users can have different views of data. This level hides the overall structure of a database system from external users. From the database designer's point of view, this level also provides information on the mapping of external records of external views to the conceptual record of conceptual view of data.

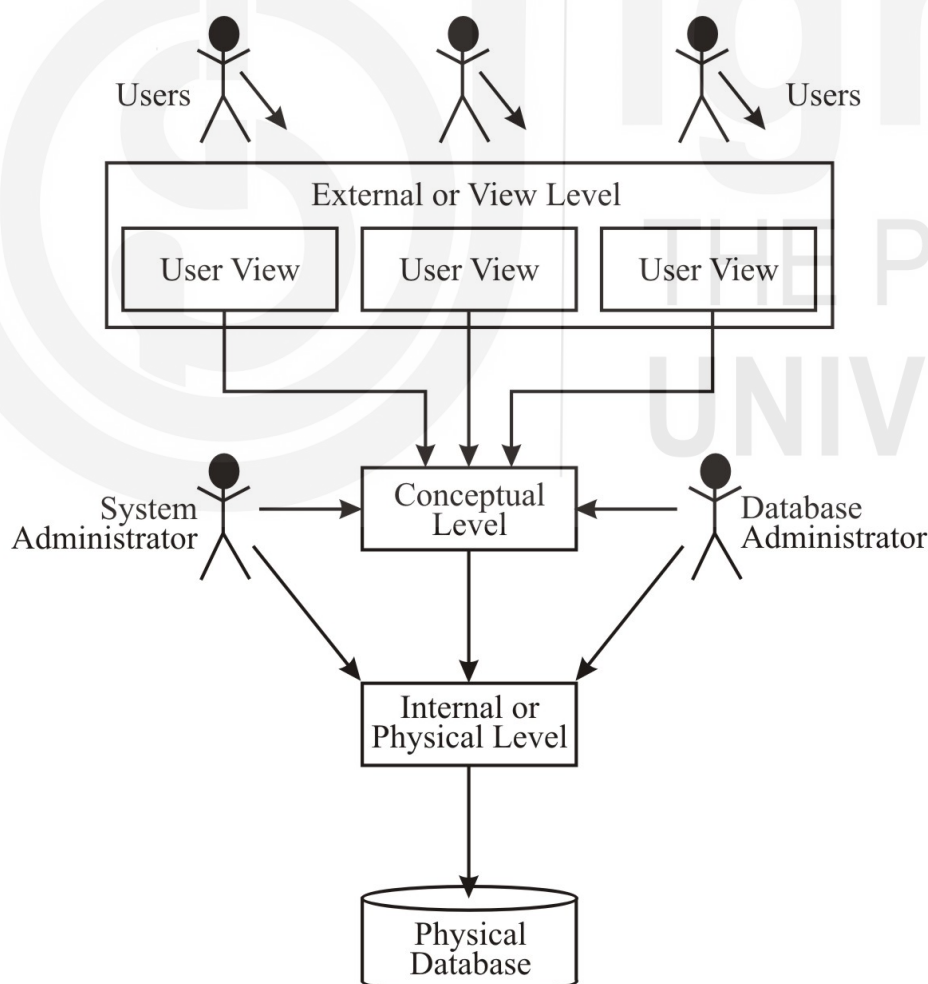


Figure 3: Logical DBMS Architecture

The Conceptual Level

The purpose of the conceptual level is to define the structure, relationships and constraints of a database system. Therefore, the conceptual level includes the structure of the data objects, relationships among those objects and the constraints on the data objects and access control of objects. A standard language called Data Definition Language (DDL) is often used in DBMSs to define the conceptual level or schema of a database system. The conceptual schema can be defined by the database administrator or the system administrator, as shown in Figure 3.

The Internal or Physical Level

A database system consists of many files, which include the data files, the meta data files, the access structure files, etc. The data files contain the actual data. The metadata files contain structure related information of data files, relationship details among data objects, constraints on data items etc. The access structure files define the control related information of the data objects. These files can be controlled by the DBMS software for access and updates. DDL can be used to describe the internal or physical schema too. In addition, this level can store additional files, which are used for enhancing the performance of the database system. These files are stored on an operating system, however, the access to these files is also under the control of the database system administrator.

1.3.2 Mappings between the three Levels and its relationship with Data Independence

The three level architecture defines a single database system across all the three levels. Therefore, different levels must map with each other. This mapping led to the concept of data independence, which was one of the major weaknesses in the file systems. This concept is explained next.

The first mapping is between the conceptual level and the external level. The external level is derived from the conceptual level. It is a part of the conceptual level, however, please note that these parts must be related else the database will lose database integrity. The advantage of this mapping is that an external user only needs to see the external level any change in the conceptual level will be hidden from the user. For example, at the conceptual level, you may keep information about the name of a person using the data items like *title*, *firstname* and *lastname*. At the external level, you may just map it to a data item *name* field. Thus, there exists a mapping, which will map:

$name = title \parallel firstname \parallel lastname$ ($\parallel$ is a concatenation operation)

Suppose, at a later point you decide to add additional data item *middlename* in the conceptual level, then you just need to change your mapping and not the programs which are based on the external level. The mapping would be changed to:

$name = title \parallel firstname \parallel middlename \parallel lastname$

Thus, the conceptual to external mapping may isolate the external users from the changes in the conceptual level. These changes can be hidden in the conceptual to internal mapping. This is also referred to as logical data independence.

The second mapping is between the physical level and the conceptual level. This mapping insulates the conceptual level from changes in file organisation, indexes etc., without changing the conceptual level. In general, such changes may be performed to enhance the performance of a database management system. For example, instead of organising the student file on enrolment number, which is the primary key, you may organise the student file on Programme + Student name. This may enhance the access time of data for many important queries to the database. The conceptual level, in this case, still remains unchanged. This is also referred to as physical data independence.

1.3.3 Objective of the three-level architecture

The three level architecture separates the data that is presented to the user from the data that is stored in the database physically. The basic objectives of three level architecture are:

- It can support different views for different users. In case of change in any application, the views related to that application are required to change. Thus, three level architecture ensures the independence of data and programs.
- As stated above, due to independence of data and application programs, the applications need not deal with physical file organisation. Therefore, the application programs and users of a database are provided with a higher level of abstraction of data. Thus, a user or application programs are not required to deal with conceptual schema and the physical schema of a database. In general, the Database Administrator (DBA) is responsible for creating and modifying the conceptual schema and physical schema using DDL.

1.4 PHYSICAL DBMS ARCHITECTURE

A Database Management System (DBMS) is a complex software, as it manages many aspects of the database, such as query languages, data integrity, data security, access to files, and performance of data access. Figure 4 shows some of the important software components of a DBMS, which should be present logically in most of the DBMSs of today.

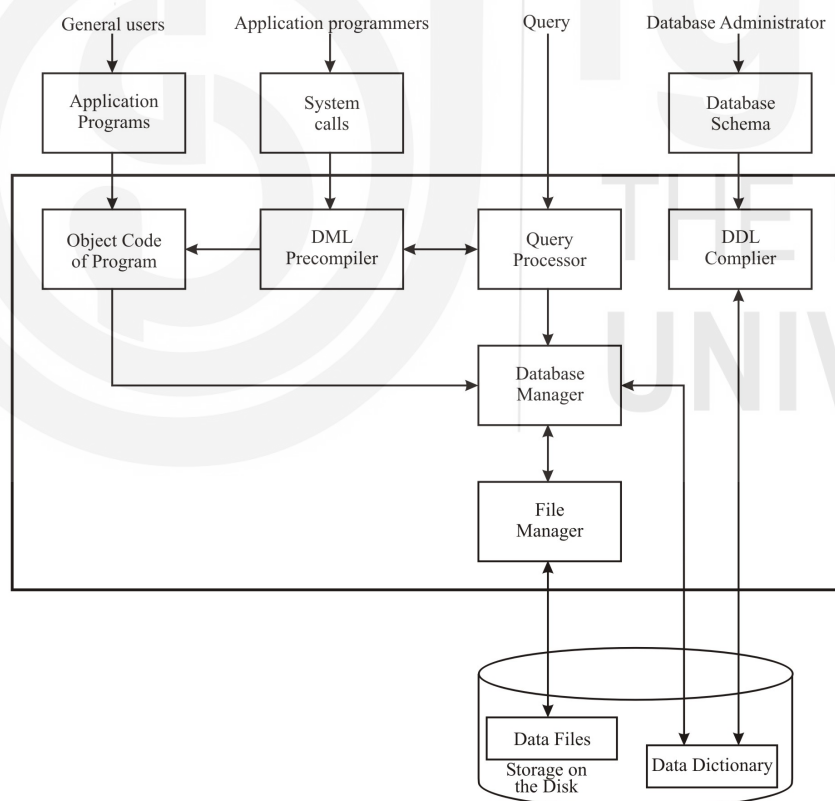


Figure 4: DBMS Structure

The software components of DBMS, as shown in Figure 4 are discussed next.

1.4.1 DML Precompiler

A DBMS consists of several languages, which are used for defining the data and for manipulation of data. For data definition, as stated earlier, a language called Data Definition Language (DDL) is used. Using DDL, you can define the structure of the data objects, data type of different data elements, integrity constraints, storage

constraints and access rights on the data. In addition, every DBMS consists of a Data Manipulation Language (DML), which is used for data insertion, modification, data access etc. The purpose of the DML precompiler is to translate the DML commands, which are written as part of application programs, into related procedure calls so that they can be executed to perform the desired operation. In addition, this component of the DBMS also coordinates the translation of queries, which are input to the query processor component.

1.4.2 DDL Compiler

A DDL compiler is used to compile a DDL statement into the related tables, which include the metadata, constraints, access rights etc. The metadata tables are stored in the data dictionary or system catalog. This metadata is used for providing information about the tables to other components of DBMS. In addition, the meta data is used for protecting data and enforcing data integrity.

1.4.3 File Manager

All the tables and metadata of the database are physically stored as files under the control of the file sub-system of the operating system. The file sub-system of the operating system has generic features, which may not be sufficient for a DBMS file. The File Manager component of the DBMS keeps track of all the data files, their index files and all other related information files of a database system. The final input/output to the database files are performed by the operating system.

1.4.4 Database Manager

Database Manager is one of the most important components of the DBMS. The role of the database manager is to accept the requests of data access or data manipulation by the user queries and application programs and perform the relevant action on the data taking the help of data dictionary and file manager. Database manager protects the data from unauthorised access and manipulation. It also enforces integrity constraints. You may please note that a database contains a large amount of data. The database manager is also responsible for deciding which data should be brought from secondary storage to main memory and back. Database manager allows simultaneous transactions on data and is also responsible for the recovery of the database in case of failures. It is also responsible for enhancing the performance of a database system access. The roles of a database manager are as follows:

- **Collaborating with file manager for file handling:** As explained earlier, the file sub-system of the operating system is responsible for storing or manipulating data and related files of a database system. The file manager component of the DBMS interacts with the operating system for this purpose. The database manager collaborates and controls the file manager for various file operations.
- **Integrity enforcement:** Data integrity relates to correctness of data in a database system. A DBMS forces several constraints on data, which allows only correct data to be stored in a database system. These constraints are stored in the data dictionary and the database manager uses a data dictionary to enforce these integrity constraints. You will learn more about integrity constraints in the later units.
- **Security enforcement:** A DBMS does not allow unauthorised users to access the data of a database. This access control can relate to the entire data or a specific item of the data. For example, as an employee of an organization, you can read only your salary information. You do not have permission to change your salary. In addition, you cannot read or change the salary of any other employee. The security constraints can also be part of the data dictionary and implemented by the database manager.
- **Backup and recovery:** The purpose of the backup and recovery component of data manager is to guard the database against the loss of data at the time of a failure of the computer system. The failure of a computer system can occur due to many different reasons, such as hardware failure communication failure

software failure etc. The database manager ensures that data of various transactions is not corrupted even after a failure.

- **Concurrency control:** One of the major uses of database systems is in transaction management. In such systems, multiple transactions access and modify the database simultaneously. The database manager ensures that consistency of data is ensured even in the presence of concurrent transactions. Transaction management is discussed in block 3 in detail.

To perform the roles or functions as discussed above, the database manager has the following software components. These components are also shown in Figure 5.

- Authorisation control is used to verify the credentials of a user to ascertain if s/he is an authorised person to access or modify the data.
- Command Processor converts the DDL/DML or other programming language commands to an executable sequence of code.
- Integrity checker checks if the data that is being inserted in a database or is being modified follows all the specified constraints on the database.
- Query Optimizer tries to optimise the processing time of different queries.
- Transaction Manager is one of the important components, which checks if the user has the right to perform a transaction.
- Scheduler component manages the consistency of data, while concurrent transactions are being processed by a database. The scheduler ensures an ordering of transaction execution, even though they may be conflicting with each other.
- Recovery Manager is responsible for the recovery of a database system to a consistent state, in case of a failure.
- Buffer Manager optimises the process of transfer of data between the main memory and the disk, where the database is stored. The buffer manager is also termed as the *cache manager*.

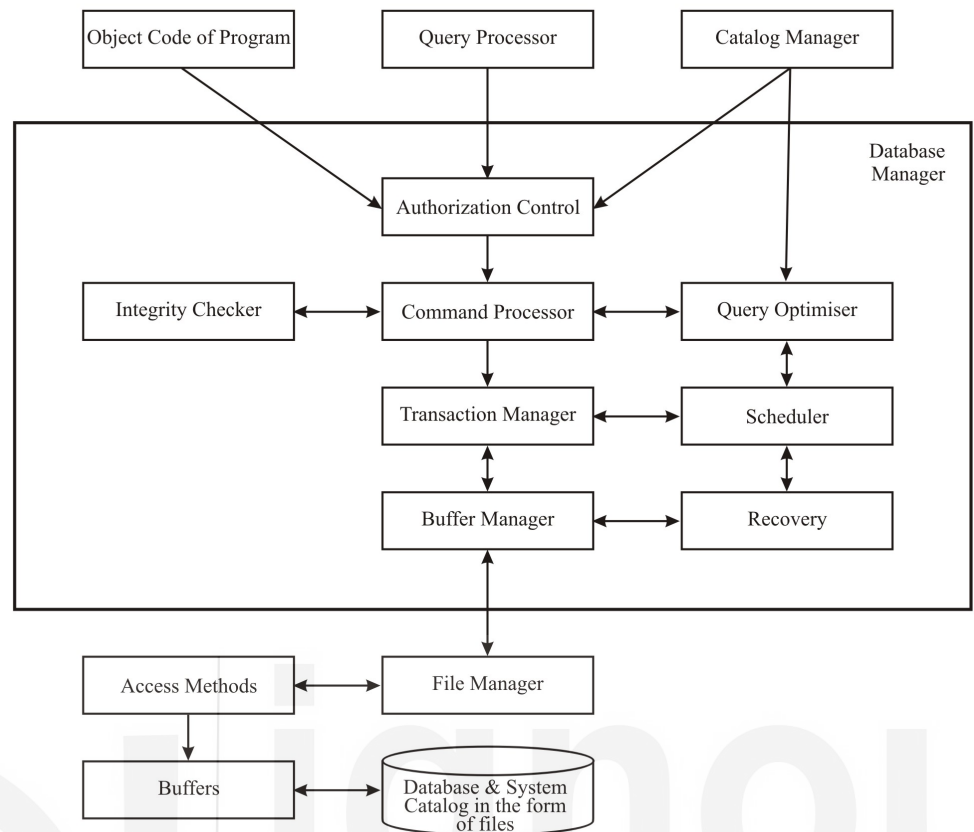


Figure 5: Components of Database Manager

1.4.5 Query Processor

Every DBMS consists of a query language which enables database creation, data manipulation, data retrieval and data control operations. This query language is usually a high level 4th generation language and is translated and optimised so that the required operations can be handled by the DBMS in the most optimal manner. In general, this query language processor consists of two basic components - the first one parses the query language into the native database language and the second optimises the query and prepares the code for query execution. The parser checks the syntax of the query, converts the query to a sequence of tasks and sends these tasks to the query optimiser. The query optimiser is responsible for generating a set of alternative query evaluation plans. It then estimates the expected total query response time, which includes access time of data from the disks, the time to execute a query and the time of communication of data or results over the network, for each of these query evaluation plans. Finally, the best query evaluation plan is selected to execute the query efficiently.

1.4.6 Database Administrator

The role of a database administrator (DBA) is to monitor the database creation (at different levels of 3-level architecture and mapping between the levels), modification, access, query response time, security, recovery in case of failure, etc. The functions of a DBA are given below:

- Defines the database Schema at different levels using DDL
- Specifies the organisation of the files and any other database access methods, including indexes, using DDL
- Performs the changes in the schema definitions, file organisation or indexes, whenever needed.
- Allows authorisation of data access or modification to different users.
- Specifies the integrity constraint on the database.

1.4.7 Data files, Indices and Data Dictionary

The data is stored in the data files. Data files can be standard files or encrypted files. These files, in general, are not accessible directly to any system user. The indices are stored in the index files. Indices provide fast access to data items. For example, a book database may be organised in the order of accession number of books yet may be indexed on Author name and Book titles for allowing faster access for searches on these attributes.

Data Dictionary: The data of an organisation is a valuable corporate resource. Commercial databases have large number of entities, attributes, data interrelationships and other information about the data. A data dictionary/directory is designed to store metadata, which is data about the data in a database system. A good data dictionary allows efficient data access, as it can provide a road map to guide users to access information within a large database. An ideal data dictionary may include the following information about a database:

- Schema definitions of internal, conceptual and external schemas, which includes:
 - the definition of every object or table
 - the attributes and their data types including primary key and attribute constraints.
 - any relationships between different attributes including foreign key constraints etc.
 - authorisation of access or modification at the level of attributes, if any.
 - Authorisations at the level of objects/tables, if any.
- Users and application programs of external schema and their permissions
- Database statistics such as number of records in every object/table, different number of occurrences of a data value in an attribute etc.

Check Your Progress 2

- 1) What are the major components of Database Manager?

- 2) Explain the functions of the person who has control of both data and programs accessing that data.

1.5 DATABASE SYSTEM ARCHITECTURES

In the present time several database system architectures are popular.

First and earliest architecture of the database system was having a centralised database system. These systems contained a database in a single machine, which was either used by a single user or were shared by several users. A single user application in the present time may be an address book of your mobile device that is being used by a single user. However, today most of the centralised database systems use multiple core processors having large memory and multiple disks, called the database servers. A large number of users are connected to these servers, mostly through remote connections. These systems fall under the category of client-server systems. At its most basic level, this client server database architecture can be broken down into two parts: the *back end* and the *front end*.

The back end are the servers, which are responsible for managing the physical database and providing the necessary support and mappings for the internal, conceptual, and external levels. Other functions of a DBMS, such as security, integrity and access control, are also the responsibility of the back end.

The front end is just any application that runs on the DBMS. These may be applications provided by the DBMS vendor, the user, or a third party. The user interacts with the front end. A front-end may be user friendly interface provided by an application developed for database system access and modifications. These applications may be developed by the vendors themselves or by software developers using an Application Programming Interface (API) or third-party applications.

In this context, many different types of interfaces and utilities are offered in support of commercial database management systems. Some of these are:

- **Interfaces:**
 - **Command line Interface:** This interface existed since the start of DBMS. You can use this interface to write DDL, DML and other permitted commands. These interfaces allow a very rich set of commands and are useful for expert users like DBA.
 - **Graphical User Interface:** Such interfaces are developed to allow users to interact through database applications using menu driven interfaces. Such interfaces have become more useful over the last decade.
- **Utilities for Backup/Restore:** The purpose of these utilities is to ensure that the current state of a database is duly backed up on secondary storage, so that it can be recovered or restored in the case of any failure or even disaster. The backup is done periodically.
- **Utilities for Load/Unload of data:** Life of a database system is quite long. The size of a database system keeps increasing if the old data is not deleted. Also, hardware wears down over a period. Therefore, a database may be required to move from one machine to another machine. Sometimes the change in DBMS software or its versions also necessitates movement of data. The Load and Upload utilities are used for the purposes as above.
- **Utilities for Reporting or Analysis:** Reports form the important component of a database systems, especially management information systems. The reports and analysis of data can be used at various levels of decision making in an organisation. These utilities analyse and report data in various visual forms for decision making.

1.6 DATA MODELS AND TRENDS

A database system requires that the data should be structured in such a manner that it allows easier data access and manipulation operations. These structures are termed as database models. A database model has to address the following issues:

- It should define a logical structure of data, so that different attributes of data are easily identified.
- It should be able to represent relationships between different data elements or objects.
- It should support constraints on data elements and data objects.

The following Table defines some of the Data Models:

Model Type	Examples
Conceptual Model: Uses entities, their attributes and relationships among entities	<i>Entity-Relationship Model:</i> This model identifies the physical or logical entities and their attributes. In addition, this model also identifies the relationships among these entities. It is mainly represented in graphical form using E-R diagrams. This is very useful in Database design. The entity relationship diagram is discussed in this Block.
Hierarchical Model: Uses data hierarchy	Hierarchical Model: It defines data as and relationships through hierarchy of data values. <i>Figure 7</i> shows an example of hierarchical model. These models are now not used in commercial DBMS products.
Record based Logical Models:	<i>Network Model:</i> Network model, as the name suggests, represents data about entities using a set of records. The relationships among these data records are represented using links. A simple network model example is explained in <i>Figure 6</i>. It shows a sample diagram for such a system. This model is a very good model as far as conceptual framework is concerned but is nowadays not used in database management systems. <i>Relational Model:</i> It models data of both entities and relationships as tables. The relationship tables are related to entity tables using a set of constraints. This model is based on a sound mathematical theory of relations. This is one of the widely used data models and will be discussed in more detail in the subsequent units of this course.
Object-based Models: Use objects as key data representation components.	<i>Object-Oriented Model:</i> An object in object-oriented model contains the data of an entity. In addition, the object also allows a set of defined operations on the data stored in an object. Further, the relationships among the objects are implemented by creating links among these objects and by implementing message passing. Object-Models are useful for databases where data interrelationships are complex, for example, Computer Assisted Design based components. These are explained in Block 4.
Semi-structured Model: Uses user defined tags	<i>Semi-structured data Model:</i> Relational model is typically called a structured model, whereas, the semi-structured model uses user defined tags, which loosely force integrity contains. However, these models are very flexible in comparison to relational models. These models are popular in web-based data management, as they are very light database systems.
Graph-based Models	<i>Graph data Model:</i> Graph is an important data structure consisting of nodes and arcs. The node can represent an entity while the links can be used to represent a relationship.

Figure 6 shows the records of students and the result of the students in various courses taken by them. All the records of the students are shown as a single file. The student records can contain one or many pointers to the course file. Please notice in Figure 6 that the person named Ajay, whose enrolment number is 21026 has obtained 50 marks in course having course code 101. Further, he has obtained 19 marks in a course

having course code 204. Thus, each record on the course is linked to one record of the student.

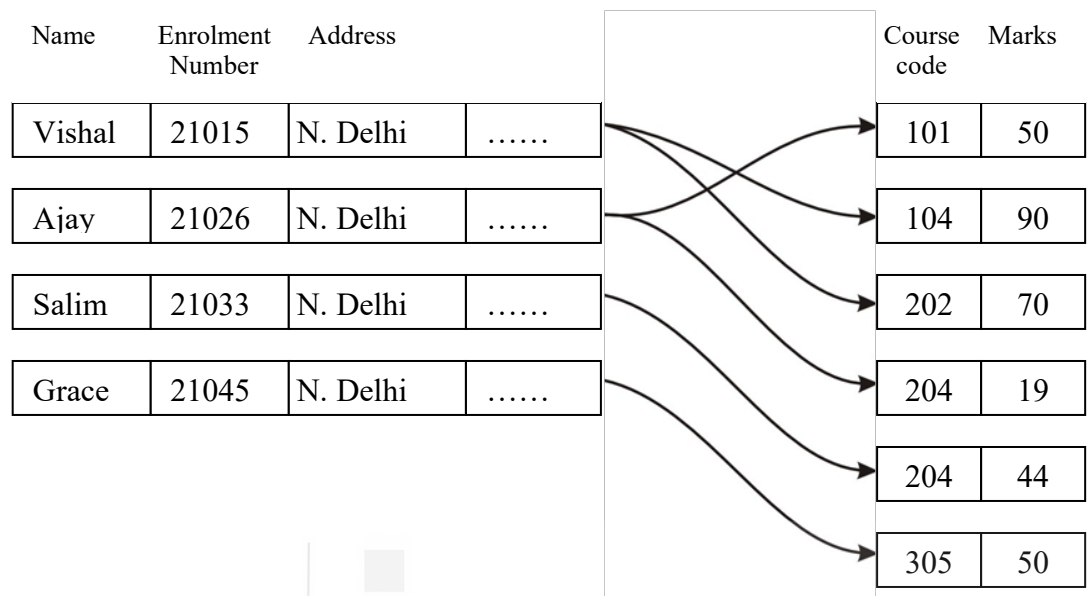


Figure 6: An example of Network Model

Figure 7 shows the data of Figure 6 in a hierarchical data model. Both these models are of historical nature. The relational model, object-oriented model and other model models are explained in the later blocks of this course.

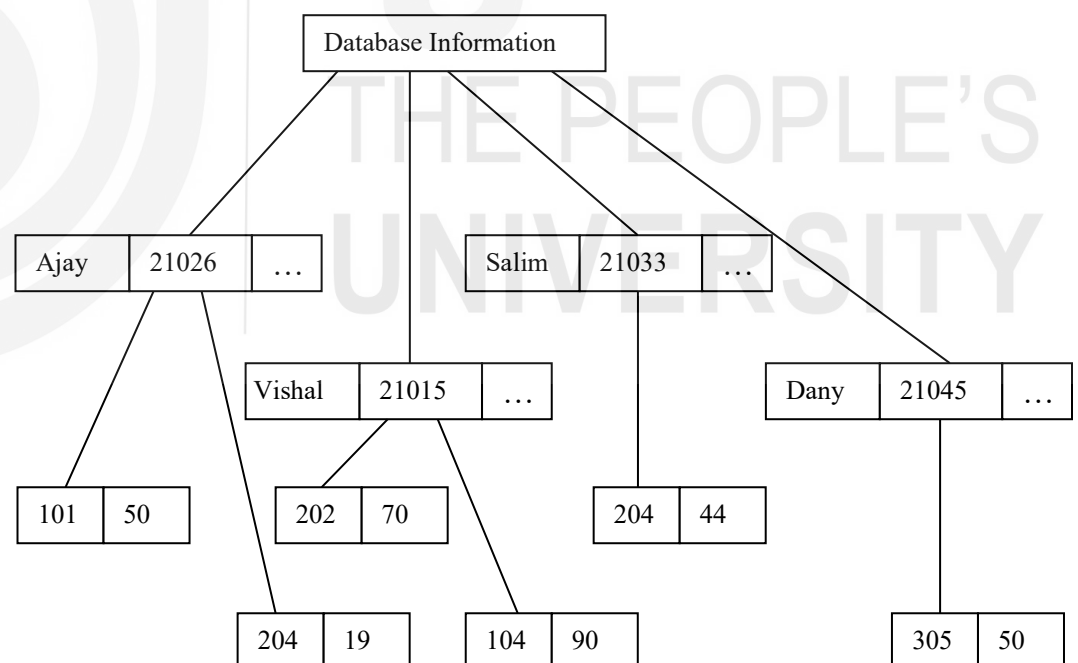


Figure 7: An example of Hierarchical Model

Relational Database management systems, which were supported by structured query language (SQL), became popular in the 1990s and later. These database systems were used to process transactions of a high-end web application. Such systems provided application users a graphical or menu driven interface using devices like computer, mobile devices and had features of user data input through keyboard, mouse, touch screens, forms, speech etc. With the growth of web applications lead to growth of semi-structured data. This led to use of eXtensible Markup Language (XML) and Java Script Object Notation (JSON) as the data exchange formats. Thus, several different database management systems, like geographical database management systems,

emerged that supported such data. With the rapid rate of growth of data newer database management systems like NoSQL databases started to grow. These databases lowered the time of application development but did not have strict control on consistency requirements. In the present time, the features of traditional and NoSQL database management systems are growing. Present day DBMSs are using cloud services for data storage and offer many database services at higher levels of abstraction leading to lesser time for database application deployment.

Check your Progress 3

- 1) What is a database backend?
.....
- 2) What is the need of utilities in a commercial database system? Explain any two such utilities.
.....
- 3) What is the difference between network and hierarchical models?
.....

1.7 SUMMARY

This unit provides you an introduction about the database system and database management system. It first defines the need of a database management system by comparing the database system approach to file system approach. The file system has several limitations due to data redundancy, however, the database approach integrates and shares the data among several applications. The unit also discusses the basic advantages of the database approach are - sharing of data, data independence, data integrity and security enforcement and transaction management for concurrent users.

The unit also explains the three-level architecture of database systems, which allows a database to be defined in terms of schemas at different levels for different types of users. Such a design allows access of relevant data to relevant users. In addition, the unit explains the physical architecture of DBMS and describes various components of DBMS. Next, the unit discusses the database system architecture for the commercial database and several data models. It also briefly presents the recent trends in DBMSs. You are advised to go through the further readings for more details on these topics.

1.8 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) DBMS manages the data of an organisation. It allows facilities for defining, updating and retrieving data of an organisation in a sharable, secure and reliable way.
- 2) The advantages of DBMS are:
 - Reduces redundancies
 - Provides environment for data independence
 - Enforces data integrity
 - Data Security
 - Answers unforeseen queries
 - Provides support for transactions, recovery etc.

3)

File Based System	Database Approach
Cheaper Data dependent Data redundancy Inconsistent Data Fixed Queries	Costly Data independent Controlled data redundancy Consistent Data Unforeseen queries can be answered

Check Your Progress 2

- 1) Integrity enforcement, control of file manager, security, backup, recovery, concurrency control, etc.
- 2) A database administrator is normally given such controls. His/her functions are: defining database, defining and optimising storage structures, and control of security, integrity and recovery.

Check Your Progress 3

1. Most of the database systems are deployed in a multi-user environment either as a centralised or distributed system. A database server, which is at the backend, manages and controls the physical data. It also provides integrity, security and access control to multiple client's requests.
2. Utilities are special purpose software, which are provided to support additional features in support of a DBMS. Two such utilities are for backup/restore – to protect data against any failure and load/unload – to load data from one hardware/software combination to a different hardware/software combination.
3. A network model stores basic entities as data records and points are used to represent the relationship between them. A hierarchical model uses almost similar model to a network model to represent data of entities but the relationship between them is implemented as a hierarchy.